

Figure 1: Implemented ROS4HRI planner stack. Dialogue, planning, deterministic execution, symbolic grounding, and AB=1 skills remain separate runtime owners. Dashed boxes mark future implementation directions rather than current execution authority.

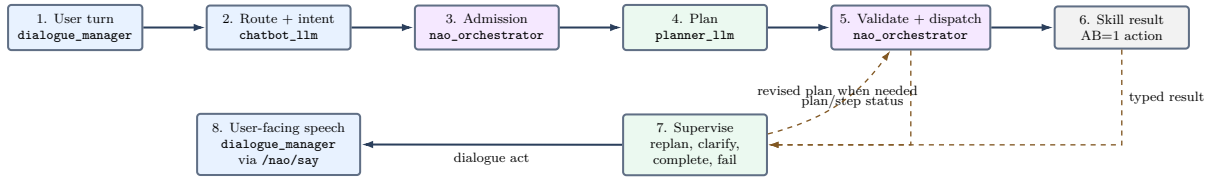


Figure 2: Planner loop used by the implemented stack. The executor reports status and evidence; it does not invent replacement plans or final wording.

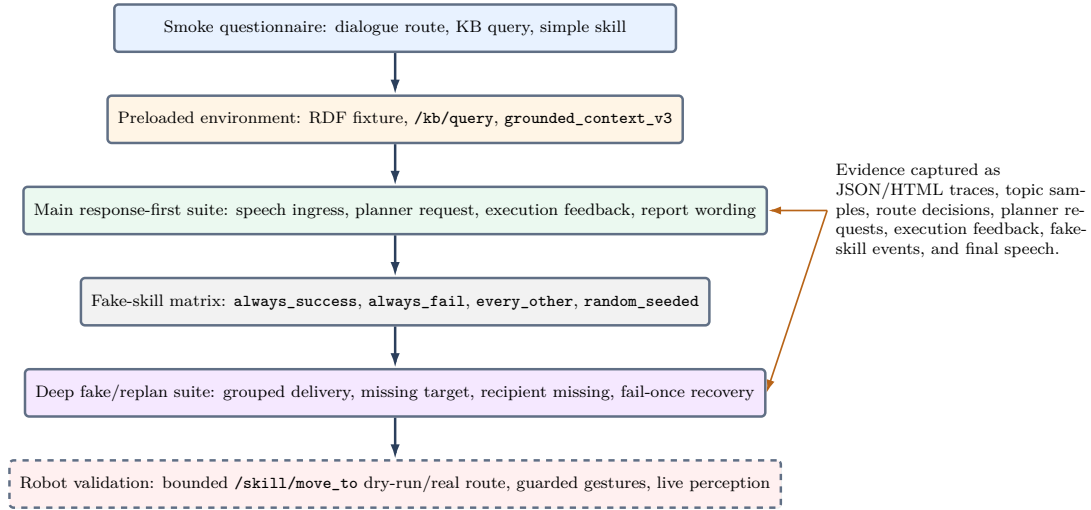


Figure 3: Validation ladder used to separate stable thesis evidence from harder replan and real-robot promotion claims.

Implementation Prose Blocks

dialogue_manager

`dialogue_manager` is the dialogue turn owner and the final speech authority. In the implemented stack, user speech enters through the ROS4HRI voice topics and is associated with a dialogue role before the chatbot backend is invoked. The node is therefore responsible for turn lifecycle, dialogue state, and speech realization, while execution-oriented decisions are delegated downstream. Planner dialogue acts return to this layer as communication requests, not as direct robot control.

This separation is important for duplicate-speech prevention. `planner_llm` can request acknowledgement, clarification, failure explanation, or completion notification through `/planner/dialogue_act`, but the user-facing utterance is realized through the dialogue/speech layer. As a result, the planner can supervise execution without becoming a speaking node, and the orchestrator can report status without inventing conversational wording.

chatbot_llm

`chatbot_llm` implements semantic turn interpretation. Its output is a route decision, a user-facing response, an intent label, target hints, and, when needed, a candidate planner request. The accepted route values are `dialogue`, `knowledge_query`, and `execution`. Dialogue-only turns and hypothetical future-action discussion stay out of the planner path. Visibility-only questions can be answered from current grounded evidence, while fresh sensing or robot-action requests are routed toward execution.

For execution turns, `chatbot_llm` does not construct executable skill plans. It sends planner-facing fields such as `goal_text`, `normalized_intents`, `scene_targets`, and `grounded_context` to the orchestrator planner gate. The current contract deliberately keeps raw detector data, full KB transport details, and hidden plan objects outside the chatbot payload. Canonical chatbot policy belongs in `src/chatbot_llm/config/chat_prompt_pack.yaml`; Python builders should provide structural framing rather than conflicting route policy.

planner_llm

`planner_llm` owns high-level task planning and goal supervision. It consumes admitted requests on `/planner/request`, builds a bounded prompt from the goal, request kind, normalized intents, compact symbolic context, skill registry, allowed step types, and output schema, then publishes executable plan envelopes on `/intents`. It also listens to `/planner/execution_feedback` and emits `/planner/dialogue_act` when the task should be clarified, failed, replanned, or reported.

The node is intentionally not a controller. It cannot directly call robot actions, write KnowledgeCore, or speak to the user. Invalid JSON, unsupported skill names, malformed arguments, or invalid recovery policies are rejected or converted into failure/clarification behavior before execution. Goal supervision is keyed by `goal_id`, with `plan_id` and `plan_version` preserving lineage across retries and replans.

nao_orchestrator

`nao_orchestrator` is the deterministic execution boundary. It receives direct intents and planner-generated plan envelopes, validates each step against the allowed registry and local execution modes, dispatches actions in order, tracks step status, and publishes structured feedback. Planner-gate mode also admits chatbot planner requests through `/nao_orchestrator/planner_request` before forwarding normalized requests to `/planner/request`.

The orchestrator is downstream-only. It can reject a plan, block unsafe continuation, dispatch supported actions, and publish `/planner/execution_feedback`; it does not generate replacement plans and does not become an LLM policy node. This is the main

safety seam between language reasoning and robot actuation. Real `walk_to` promotion is deliberately narrow: the real route maps to `/skill/move_to` with bounded body-relative `x_m`, `y_m`, and `theta_rad`; semantic target navigation remains fake or future decomposition work.

Grounding and Knowledge

The grounding path converts perception and KnowledgeCore state into compact symbolic evidence. `nao_scene_grounding` owns detector normalization, detector-derived object facts, optional spatial overlay merging, and `/scene/summary`. `kb_skills` owns the query/revise transport boundary to KnowledgeCore. `chatbot_llm` projects those sources into `grounded_context_v3`, whose main fields are `entities`, `locations`, bounded relations, counts, and freshness metadata.

The planner receives this compact T0 context for reasoning, but the context is not treated as proof that an execution-time effect succeeded. `AB=1` skills remain responsible for fresh evidence at execution time. This distinction prevents image-plane detections, stale KB facts, or planning-time summaries from being reported as completed robot actions.

Fake Skills and Validation Seams

`fake_skills` provides deterministic `AB=1` action servers for validation without requiring the physical robot for every experiment. The fake-skill server can run scenario-based or global policies such as `always_success`, `always_fail`, `every_other`, and `random_seeded`. It publishes fake-skill events and returns structured result payloads, including success/failure mode and optional KB effects.

This layer is not a replacement for robot validation. It is a controlled seam for testing route correctness, registry exposure, plan validation, execution feedback, clarification, safe failure, and replanning. The deep fake/replan suite should therefore be reported separately from the main response-first runtime score.

Models and Runtime Configuration

The runtime behavior depends on launch profile, LLM endpoint, prompt pack, planner skill registry, grounding mode, and fake/real skill execution flags. The active launch documentation separates simulator, robot, demo, ASR-only, fake-skill, and object-grounding profiles. For thesis validation, record the exact launch arguments, model names, prompt-pack commit, fake-skill policy, and artifact paths for each run.

Validation Section Seed

Validation Strategy

The validation is organized as a ladder rather than a single demo. First, smoke and main response-first runs test dialogue routing, KB queries, simple skills, composite skills, execution feedback, and final speech. Second, preloaded-environment runs inject deterministic KnowledgeCore scenes and verify that the same objects appear in `/kb/query`, `grounded_context_v3`, planner requests, execution feedback, and user-facing answers.

Third, fake-skill and deep fake/replan runs stress controlled failure, missing targets, missing recipients, grouped-location delivery, and post-skill KB effects. Finally, real-robot promotion is evaluated separately for bounded locomotion and guarded gestures.

Evidence class	Acceptance signal
Dialogue routing	Dialogue and future-action discussion do not create planner requests. Exactly one user-facing utterance is produced.
Grounded context	Inserted or preloaded KB facts appear in <code>/kb/query</code> , <code>grounded_context_v3</code> , and the chatbot/planner turn trace.
Plan validity	<code>planner_llm</code> emits valid JSON under <code>Intent.data.plan</code> ; unsupported skills and malformed arguments are rejected.
Execution feedback	<code>nao_orchestrator</code> publishes <code>plan_accepted</code> , <code>step_started</code> , <code>step_succeeded/step_failed</code> , and terminal plan status with lineage.
Recovery behavior	Fake failures cause truthful failure, clarification, or replan behavior without false completion.
Robot promotion	Real routes remain operator-gated and bounded; dry-run and physical routes are scored separately.

Table 1: Validation evidence classes for the thesis implementation.